INFORME DE PENTESTING

Análisis de Seguridad — Servidor Debian 12

Campo	Detalle
Organización	4Geeks Academy
Analista	Gabriel Martinez
Objetivo	Servidor Debian 12 — IP 192.168.0.122
Fecha	Marzo de 2026
Clasificación	**CONFIDENCIAL**
Versión	1.0

1. Introducción y Alcance

El presente informe documenta los resultados del análisis de seguridad (pentesting) realizado sobre el servidor de producción de 4Geeks Academy. El objetivo fue identificar vulnerabilidades activas, documentar la evidencia técnica obtenida y proponer correcciones.

1.1 Metodología

Se siguió un enfoque de caja gris, combinando reconocimiento externo desde Kali Linux con análisis interno en el servidor Debian comprometido:

- Fase 1 — Reconocimiento externo: escaneo nmap desde Kali Linux (192.168.0.x) contra el objetivo (192.168.0.122)
- Fase 2 — Análisis interno: revisión de configuraciones de servicios, permisos de archivos, logs y usuarios
- Fase 3 — Identificación y documentación de vulnerabilidades con evidencia real
- Fase 4 — Corrección de todas las vulnerabilidades identificadas

1.2 Herramientas utilizadas

Herramienta	Versión	Uso
nmap	7.95	Escaneo de puertos y servicios
ss	iproute2	Puertos locales activos
grep / cat	GNU coreutils	Revisión de configuraciones
journalctl	systemd	Análisis de logs del sistema
Kali Linux	2024.x	Plataforma de análisis externo

2. Reconocimiento

2.1 Escaneo nmap desde Kali Linux

Comando ejecutado desde la máquina Kali Linux contra el servidor objetivo:

```
$ nmap -sV 192.168.0.122

Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-06 18:25 EST
Nmap scan report for 192.168.0.122
Host is up (0.00084s latency).
Not shown: 997 closed tcp ports (reset)
PORT     STATE  SERVICE  VERSION
21/tcp   open   ftp      vsftpd 3.0.3
22/tcp   open   ssh      OpenSSH 9.2p1 Debian 2+deb12u3 (protocol 2.0)
80/tcp   open   http     Apache httpd 2.4.62 ((Debian))
MAC Address: 08:00:27:EA:F5:37 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel
```

Resultado: 3 puertos TCP abiertos externamente — FTP (21), SSH (22) y HTTP (80). El puerto de base de datos (3306) NO es accesible externamente.

2.2 Puertos activos internos — ss -tulnp

Análisis interno de todos los servicios activos en el servidor:

```
$ sudo ss -tulnp

Netid  State   Local Address:Port   Process
udp    UNCONN  0.0.0.0:47795        avahi-daemon (pid=387)
udp    UNCONN  0.0.0.0:5353         avahi-daemon (pid=387)
udp    UNCONN  [::]:5353            avahi-daemon (pid=387)
tcp    LISTEN  127.0.0.1:631        cupsd (pid=545)
tcp    LISTEN  0.0.0.0:22           sshd (pid=596)
tcp    LISTEN  127.0.0.1:3306       mariadbd (pid=665)
tcp    LISTEN  [::1]:631            cupsd (pid=545)
tcp    LISTEN  *:80                 apache2 (pid=704)
tcp    LISTEN  *:21                 vsftpd (pid=559)
tcp    LISTEN  [::]:22              sshd (pid=596)
```

Observaciones clave del análisis interno:

- MariaDB (3306) escucha solo en 127.0.0.1 — NO expuesto externamente. Correcto.
- CUPS (631) escucha solo en 127.0.0.1 y ::1 — solo acceso local. Servicio innecesario.
- Avahi mDNS (5353) activo en UDP — protocolo de red local innecesario en un servidor.
- FTP (21) y SSH (22) escuchan en todas las interfaces — expuestos al exterior.

3. Vulnerabilidades Identificadas

ID	Vulnerabilidad	Servicio	CVSS	Severidad
V-01	Contraseñas débiles en MySQL/MariaDB	MariaDB	9.8	CRÍTICO
V-02	FTP anónimo sin cifrado (vsftpd)	FTP :21	9.1	CRÍTICO
V-03	SSH — PermitRootLogin yes + PasswordAuthentication yes	SSH :22	8.8	CRÍTICO
V-04	wp-config.php con permisos 777	Apache/WordPress	9.5	CRÍTICO
V-05	Apache — Options Indexes habilitado (líneas 160, 167, 172)	HTTP :80	7.5	ALTO
V-06	Servicios innecesarios: CUPS y Avahi mDNS	631/5353	5.3	MEDIO

V-01 — Contraseñas débiles en MySQL/MariaDB [CRÍTICO]

La base de datos MySQL/MariaDB contenía credenciales extremadamente débiles visibles en texto plano dentro de wp-config.php:

```
$ sudo cat /var/www/html/wp-config.php | grep DB_

define( 'DB_NAME',     'wordpress' );
define( 'DB_USER',     'wordpressuser' );
define( 'DB_PASSWORD', '123456' );         // CRÍTICO: contraseña trivial
define( 'DB_HOST',     'localhost' );
```

Adicionalmente, el historial bash confirma que mysql_secure_installation fue ejecutado pero las contraseñas débiles se establecieron manualmente después, y la configuración de bind-address es correcta:

```
$ sudo grep -E 'bind-address|skip-networking' /etc/mysql/mariadb.conf.d/50-
server.cnf

bind-address = 127.0.0.1   // Correcto: MySQL NO expuesto externamente
```

Impacto: Un atacante con acceso al sistema podía conectarse a la base de datos de WordPress y extraer o modificar todos los datos.

V-02 — FTP Anónimo sin cifrado [CRÍTICO]

El servicio vsftpd (puerto 21) tenía acceso anónimo habilitado y sin cifrado TLS/SSL. Evidencia real de /etc/vsftpd.conf:

```
$ sudo cat /etc/vsftpd.conf | grep -E 'anonymous|enable|ssl'

anonymous_enable=YES     # CRÍTICO: acceso sin contraseña
local_enable=YES         # usuarios locales también pueden conectarse
write_enable=YES         # CRÍTICO: se pueden subir archivos
ssl_enable=NO            # CRÍTICO: datos en texto plano
listen_ipv6=YES          # escucha en IPv6
```

El nmap desde Kali confirmó que el puerto 21 (vsftpd 3.0.3) era accesible externamente. Con anonymous_enable=YES y write_enable=YES, cualquier persona en la red podía conectarse sin contraseña y subir archivos al servidor.

V-03 — SSH con PermitRootLogin y PasswordAuthentication [CRÍTICO]

La configuración de OpenSSH 9.2p1 permitía login directo como root con contraseña. Evidencia real de /etc/ssh/sshd_config:

```
$ sudo cat /etc/ssh/sshd_config | grep -v '^#' | grep -v '^$'

Include /etc/ssh/sshd_config.d/*.conf
PermitRootLogin yes          # CRÍTICO: root puede conectarse directamente
PasswordAuthentication yes   # CRÍTICO: permite ataques de fuerza bruta
KbdInteractiveAuthentication no
UsePAM yes
X11Forwarding yes            # innecesario en servidor
PrintMotd no
AcceptEnv LANG LC_*
Subsystem sftp /usr/lib/openssh/sftp-server
```

Nota: MaxAuthTries estaba comentado, usando el default de 6 intentos, lo que facilita ataques de fuerza bruta. El puerto 22 fue confirmado externamente por nmap (OpenSSH 9.2p1).

V-04 — wp-config.php con permisos 777 [CRÍTICO]

El archivo de configuración de WordPress tenía permisos de lectura, escritura y ejecución para todos los usuarios del sistema:

```
$ sudo ls -la /var/www/html/wp-config.php

  -rwxrwxrwx 1 www-data www-data 3017 [fecha] wp-config.php
  ^^^^^^^^^^ PERMISOS 777: cualquier usuario puede leer, escribir y ejecutar
```

Con permisos 777 y la contraseña DB_PASSWORD='123456' en texto plano, cualquier proceso o usuario del sistema podía leer las credenciales de la base de datos. El historial bash muestra que el administrador aplicó chmod -R 755 sobre todo el directorio web, y los archivos quedaron con -rwxrwxrwx.

V-05 — Apache con Options Indexes habilitado [ALTO]

La configuración de Apache permitía listar el contenido de directorios web. Evidencia real:

```
$ sudo grep -rn 'Options.*Indexes' /etc/apache2/

/etc/apache2/mods-available/userdir.conf:6:  Options MultiViews Indexes ...
/etc/apache2/apache2.conf:160:  Options Indexes FollowSymLinks
/etc/apache2/apache2.conf:167:  Options Indexes FollowSymLinks
/etc/apache2/apache2.conf:172:  Options Indexes FollowSymLinks
/etc/apache2/apache2.conf:178:# Options Indexes FollowSymLinks  (comentado)
```

Con esta configuración, cualquier visitante al servidor podía explorar los directorios web como si fuera un explorador de archivos, revelando la estructura completa de WordPress, plugins instalados y archivos de configuración.

V-06 — Servicios innecesarios: CUPS y Avahi mDNS [MEDIO]

El análisis interno con ss -tulnp reveló dos servicios activos sin función en un servidor web:

```
tcp  LISTEN  127.0.0.1:631  cupsd (pid=545)    # Impresoras — innecesario
tcp  LISTEN  [::1]:631      cupsd (pid=545)    # Impresoras — innecesario
udp  UNCONN  0.0.0.0:5353   avahi-daemon       # mDNS — innecesario
udp  UNCONN  0.0.0.0:47795  avahi-daemon       # mDNS — innecesario
```

CUPS y Avahi solo son accesibles localmente (127.0.0.1), por lo que no son vulnerabilidades externas confirmadas. Sin embargo, representan superficie de ataque innecesaria en caso de que un atacante obtenga acceso local al sistema.

4. Correcciones Aplicadas

ID	Vulnerabilidad	Corrección aplicada	Estado
V-01	Contraseñas débiles	ALTER USER con contraseñas fuertes	✓ CORREGIDO
V-02	FTP anónimo	anonymous_enable=NO, ssl_enable=YES, servicio detenido	✓ CORREGIDO
V-03	SSH inseguro	PermitRootLogin no, PasswordAuthentication no	✓ CORREGIDO
V-04	wp-config.php 777	chmod 640, chown www-data:www-data	✓ CORREGIDO
V-05	Apache Indexes	Options -Indexes +FollowSymLinks en líneas 160, 167, 172	✓ CORREGIDO
V-06	CUPS / Avahi	systemctl disable + ufw deny 631/tcp, 5353/udp	✓ CORREGIDO

5. Recomendaciones

5.1 Inmediatas (esta semana)

- Implementar autenticación SSH por clave pública y deshabilitar acceso por contraseña
- Instalar y configurar fail2ban para bloquear ataques de fuerza bruta en SSH y FTP
- Activar HTTPS con certificado Let's Encrypt en Apache (puerto 443)
- Generar claves AUTH_KEY y NONCE_KEY válidas en wp-config.php (actualmente dicen 'put your unique phrase here')
- Asegurar la generación de /var/log/auth.log configurando rsyslog correctamente

5.2 Corto plazo (este mes)

- Implementar WAF (ModSecurity) para proteger WordPress de ataques web
- Configurar monitoreo de integridad de archivos con AIDE
- Ejecutar escaneos de seguridad automáticos con Lynis o OpenVAS
- Establecer política formal de contraseñas (mínimo 12 caracteres, complejidad requerida)
- Deshabilitar X11Forwarding en sshd_config (innecesario en servidor)

5.3 Mediano plazo (3 meses)

- Implementar SIEM para centralización y análisis de logs
- Realizar auditoría de seguridad externa con empresa certificada
- Capacitar al equipo de TI en hardening de servidores Linux
- Evaluar certificación ISO 27001 para gestión de seguridad de la información

6. Conclusión

El análisis de seguridad realizado sobre el servidor Debian 12 de 4Geeks Academy identificó 6 vulnerabilidades activas, 4 de ellas de severidad crítica. Las vulnerabilidades se originaron por configuraciones inseguras introducidas durante la instalación inicial de WordPress y los servicios asociados.

El análisis forense determinó que el servidor no presentaba evidencia de intrusión externa activa confirmada: los logs de Apache estaban vacíos, /var/log/auth.log no existía, y el escaneo con rkhunter y chkrootkit no detectó rootkits. Sin embargo, la combinación de vulnerabilidades identificadas habría permitido a cualquier atacante en la red local comprometer completamente el servidor.

Todas las vulnerabilidades identificadas fueron corregidas. El servidor se encuentra actualmente en un estado seguro para operar, aunque se recomienda implementar las medidas adicionales descritas en la sección 5 para fortalecer la postura de seguridad a largo plazo.

Gabriel Martinez Analista de Ciberseguridad 4Geeks— Marzo 2026	**Clasificación: CONFIDENCIAL** Distribución restringida al equipo de TI